

useRef Hook

- The useRef Hook is a versatile tool, often overlooked but crucial for managing references, optimizing performance, and interacting directly with the Document Object Model (DOM).
- The useRef() Hook is a built-in React feature that persists values between component re-renders.
- Unlike state variables managed by useState, values stored in a ref object remain unchanged across renders, making it ideal for scenarios where data doesn't directly affect the UI but is essential for the component's behavior.
- When React encounters a useRef() Hook, it returns a plain JavaScript object with a single property: current.
- This current property stores mutable values, which can be of any type, from simple values like numbers and strings to complex objects, functions, or even references to DOM elements.
- React assigns the initial value you define to the current property of the returned reference.
- React will set the value of the useRef to undefined if you don't provide an initial value.
- Importantly, you can update this current value directly without triggering a re-render of the component.

Use Cases

1. Accessing DOM Elements (Like Focusing an Input)

- One of the most common uses of useRef is to directly interact with HTML elements for example, focusing on an input box when the page loads.
- You simply connect the input to useRef and tell it to focus.
- This is helpful when you want to guide the user to start typing immediately or scroll to a specific section of the page.

2. Storing Values Without Causing Re-renders

- Sometimes you need to store a value temporarily in your component, but you don't want that value to trigger a UI change or re-render the page. useRef allows you to do this.

- It keeps the value updated internally, but the screen doesn't change because of it. Think of it like writing on a sticky note that only you can see behind the scenes.

3. Tracking Previous Values

- You might want to remember what a value used to be before it changed like tracking the previous name a user typed.
- `useRef` can hold this previous value for you, even after the current value changes.
- It's great when you want to compare the current and previous states to make decisions or display differences.

4. Handling Timers and Intervals

- When working with timers or `setInterval` functions, you usually get an ID number to clear or stop it later.
- `useRef` helps you store that timer ID safely so you can cancel or reset the timer whenever you need.
- Without `useRef`, managing these timers would be tricky, especially during cleanup when components disappear.

5. Scrolling to a Section of the Page

- You can also use `useRef` to scroll smoothly to a particular section of the page.
- This is useful when you have a "Go to Top" button or navigation that jumps to a specific section.
- You just tell the browser to scroll to the element connected to the `useRef`.

6. Keeping Values Between Renders (like Render Count)

- If you want to keep track of something like how many times a component has rendered, `useRef` is a perfect choice.
- Each time the component updates, you can increase that value and since `useRef` doesn't trigger re-renders, it quietly keeps track in the background.

Example 1 - Focus an Input on Button Click

```

import React from 'react'
import { useRef } from 'react'

function Ex1() {
  const inputRef = useRef(null)
  const handleFocus={() => {
    inputRef.current.focus()
  }}
  return (
    <div>
      <input type="text" ref={inputRef} placeholder="Enter your name"/>
      <button onClick={handleFocus}>Focus on Input</button>
    </div>
  )
}

export default Ex1

```

- inputRef.current holds a reference to the actual HTML <input> element.
- The .focus() method is a built-in browser method to focus input fields.
- Without useRef, it's difficult to manipulate elements directly in functional components.

Example 2 - Auto-Focus Input When Component Loads

```

import React from 'react'
import { useEffect } from 'react'
import { useRef } from 'react'

function Ex2() {
  const inputRef = useRef(null)
  useEffect(() => {
    inputRef.current.focus()
  })
}

```

```

    }, [])

    return (
      <div>
        <input type="text" ref={inputRef} placeholder="Enter a name"/>
      </div>
    )
  }
}

export default Ex2

```

- `useEffect(() => { ... }, [])` runs once when the component loads.
- `inputRef.current.focus()` is called immediately after the input is added to the page.
- Great for login screens, search bars, or form pages where you want users to start typing immediately.

Example 3: Multiple Inputs – Focus Next Input

```

import React from 'react'
import { useRef } from 'react'

function Ex3() {
  const firstRef = useRef(null)
  const secondRef = useRef(null)

  const handleFocus = ()=>{
    secondRef.current.focus()
  }

  return (
    <div>
      <input type="text" ref={firstRef} placeholder="First Name"/>

```

```

    <input type="text" ref={secondRef} placeholder="Second Name" />
    <button onClick={handleFocus}>Go to the last name</button>

  </div>
)
}

export default Ex3

```

- You can have multiple refs, each pointing to different inputs.
- The button click moves focus from first input to second input.

Example 4: Storing Mutable Data Without Causing Re-renders

- Sometimes we need to store values temporarily, like counters, IDs, or temporary flags.
- But if we store them in useState, it will re-render the whole component, which is unnecessary.

```

import React from 'react'
import { useRef } from 'react'

function Ex4() {
  const countRef = useRef(0)

  const handleClick = () =>{
    countRef.current ++
    alert(`You clicked the button ${countRef.current}`)
  }

  return (
    <div>
      <button onClick={handleClick}>Click Me</button>
    </div>
  )
}

```

```
}
```

```
export default Ex4
```

- This React component uses the useRef hook to count how many times a button is clicked without causing the component to re-render each time.
- Inside the component, a reference variable countRef is initialized with a value of 0.
- When the button is clicked, the handleClick function runs — it increases countRef.current by one and displays an alert showing how many times the button has been clicked.
- The count is not shown on the screen, only in the alert box.
- useRef is used here instead of useState because we don't need to update the UI every time the value changes.
- It simply keeps track of the number in the background without triggering a re-render.
- This makes useRef ideal for storing values that change but don't need to visually update the screen, such as counters, timers, or tracking previous values quietly behind the scenes.

Example 5: Storing Previous State Value

- React doesn't automatically keep the previous value of a state, so if you want to compare old and new values, useRef helps.

```
import React from 'react'
import { useEffect } from 'react'
import { useRef } from 'react'
import { useState } from 'react'

function Ex5() {
  const [score, setScore] = useState(0)
  const prevScoreRef = useRef()
```

```

useEffect(()=>{
  prevScoreRef.current = score
}, [score])
return (
  <div>
    <p>Current Score: {score}</p>
    <p>Previous Score: {prevScoreRef.current}</p>
    <button onClick={()=>setScore(score+1)}>Increase Score</button>
  </div>
)
}

export default Ex5

```

- This React component demonstrates how to use the useRef and useEffect hooks together to keep track of the previous value of a state variable.
- The component imports useState, useEffect, and useRef, and starts by creating a state variable called score, initially set to 0.
- It also creates a ref called prevScoreRef to store the previous value of the score.
- Inside the useEffect, whenever the score changes, the current value of score is saved into prevScoreRef.current.
- This way, prevScoreRef.current always holds the value of the score before the last update.
- In the UI, it displays both the current score and the previous score, and has a button to increase the score by 1.
- This is a useful technique when you want to compare the current and previous values of a state variable without triggering extra re-renders.

Example 6: Scroll to Element on Button Click

- We use useRef to allow smooth scroll to a specific section of the page.
import React from 'react'

```

import { useRef } from 'react';

function Ex6() {
  const boxRef = useRef();
  const scrollToBox = () => {
    boxRef.current.scrollIntoView({ behaviour: 'smooth' })
  }
  return (
    <div>
      <button onClick={scrollToBox}>Click to Scroll</button>
      <div style={{ height: "1200px" }}></div>
      <div ref={boxRef} style={{ height: "100px", background: "yellow" }}>
        Hello! I'm the box.
      </div>
    </div>
  )
}

export default Ex6

```

- `useRef()` is used to create a reference called `boxRef`. This reference will later point to the box (a `div` element) that we want to scroll to.
- The `scrollToBox` function is triggered when the button is clicked. Inside this function, we use `boxRef.current.scrollIntoView({ behavior: "smooth" })`.
- This tells the browser to scroll the page to the element assigned to `boxRef`, and do it smoothly.
- `scrollIntoView()` is a browser feature that tells the browser to scroll the page so that the element becomes visible in the viewport.

- In the return section, there is a button labeled "Scroll to Box", followed by a big empty space (`<div style={{ height: "1000px" }}>`) that pushes the box further down the page so it's not visible initially.
- Finally, the box element (`<div ref={boxRef}>`) is where `useRef` connects.
- This means that when you click the button, the browser knows exactly which box to scroll to, because `boxRef` is directly linked to that element.